

Computational Thinking Mini Project

WHO AM I: BUILDING A LITERARY CHARACTER CHATBOT

Creator: Vimala Telu

About This Resource

This mini-project serves as a ready-to-use instructional resource that integrates computational thinking and literary analysis by guiding Grade 10 students in designing a Scratch-based literary character chatbot. Students apply sequences, loops, events, conditionals, functions, and parameters to encode character motivations and textual evidence into interactive dialogue. The project deepens analysis of CCSS RL.9-10.3 through active learning, peer testing, and iterative design.

Target Audience:

Grade Level	Grade 10 English Language Arts (ages 15–16)
Course Context	Standard or Honors ELA — character development and literary analysis unit
Learner Profile	Students with no previous programming experience. Familiarity with a class novel or literary text is required.
Class Size	20–30 students
Duration	3 class periods × 50 minutes (150 minutes total)
Adaptability	Suitable for Grades 9–12. The CT logic, active learning structure, and assessment criteria apply to any literary text, grade level, or ELA sub-topic.

Standard(s) Addressed:

Primary	CCSS.ELA-LITERACY.RL.9-10.3 — Analyze how complex characters, such as those with multiple or conflicting motivations, develop throughout a text, interact with other characters, and drive the plot or deepen the theme.
Supporting	CCSS.ELA-LITERACY.RL.9-10.1 — Cite strong and thorough textual evidence to support analysis of what the text explicitly states as well as inferences drawn from it.
Technology	Students with no previous programming experience. Familiarity with a class novel or literary text is required.

Objectives of the Project:

Knowledge & Skills: Learners will use sequences, loops, abstraction, events, conditionals, functions, and parameters—the fundamental programming concepts of this unit—to design and develop a functioning literary-character chatbot in Scratch.

Experience: Learners will develop communication, critical thinking, evidence-based reasoning, collaborative inquiry, and iterative design skills through structured group analysis, pair programming, and peer feedback sessions.

Learning Objectives — Bloom's Taxonomy

Each objective below aligns with a level of Bloom's Revised Taxonomy, moving from basic knowledge to innovative synthesis.

Bloom's Level	Cognitive Skill	Learning Objective
Remember	Recall	Learners will identify and name at least six CT/programming concepts (sequences, loops, abstraction, events, conditionals, functions, and parameters) and recall their definitions.
Understand	Explain	Learners will explain how each programming concept maps to a literary analysis skill.
Apply	Implement	Learners will apply events, conditionals, functions, and parameters in Scratch to build a chatbot that responds to thematic questions about a literary character using direct textual evidence.
Analyze	Examine	Learners will analyze how a literary character's motivations, conflicts, and development across a text can be encoded as conditional logic branches grounded in accurately cited quotations.
Evaluate	Justify	Learners will assess the authenticity and accuracy of a peer's chatbot responses against the source text and offer evidence-based feedback using both CT vocabulary and literary terminology.
Create	Design	Learners will create and develop an original, fully operational literary character chatbot in Scratch that incorporates all essential CT concepts and shows in-depth, evidence-based character analysis.

Problem/Challenge Identification:

A persistent challenge in secondary ELA is that students can describe what a literary character does, such as summarizing plot actions and identifying surface traits, but often struggle to analyze who the character truly is, including their conflicting motivations, psychological contradictions, and the deliberate craft the author uses to develop them throughout the text. CCSS RL.9-10.3 explicitly asks students to analyze characters with "multiple or conflicting motivations," yet traditional essay formats often only allow surface-level responses that describe rather than interpret. This project addresses that challenge through a computational approach. By requiring students to program a chatbot that speaks in a character's authentic voice and responds to thematic questions with direct citations, the project makes superficial understanding structurally impossible. A chatbot whose responses are vague, inaccurate, or out of voice immediately fails when tested by real readers during peer beta-testing. The design challenge promotes genuine analytical depth. At the same time, the project links literary learning to one of the most influential technologies of our time. Every AI chatbot, virtual assistant, and FAQ system is built on the same core logic: events trigger responses, conditionals branch the logic, functions make the code reusable, and parameters customize the output. By creating a literary chatbot, learners become informed, critical participants in an AI-driven world, not just its users.

The Design Challenge for Learners:

A class has been studying a complex literary character whose motivations, conflicts, and development reveal the author's deeper themes.

The challenge: build an interactive chatbot in Scratch that speaks like this character. When a reader types a question about one of the character's key themes, your chatbot must respond in the character's authentic voice using a direct quotation from the text as evidence.

The chatbot will be tested by their classmates — it must be **character-accurate, evidence-grounded, and analytically defensible**.

Computational Thinking/Programming Concepts:

This project combines **six programming concepts** that together meet the analytical needs of character analysis. Each concept serves as both a technical element of the Scratch chatbot and a thinking tool that enhances literary interpretation.

Programming Concept	Description	Action Point	Explanation with reference to the Subject
Sequences	A sequence of steps that must be followed in an exact, unchangeable order.	Every chatbot interaction follows a strict sequence: receive input → evaluate condition → match keyword → call function → display response. Interrupting this order breaks the chatbot.	A character's development unfolds in sequence — the same trait carries different meanings in Act 1 versus Act 5. Analyzing the sequence of events shows how the author constructs change over time.
Loops	A set of instructions that repeats automatically while a condition stays true.	A "forever" loop keeps the chatbot listening for new questions nonstop. Without it, the chatbot answers only once and then stops forever.	Deep literary analysis is an iterative process. Skilled readers repeatedly revisit the text with new questions. This cycle reflects sustained engagement with a character.
Abstraction	Simplifying by removing unnecessary details to develop a solution that applies generally.	A single reusable "character voice" function abstracts the response format from the content — one template works for any thematic topic. Change the format once; all responses update automatically.	Pattern recognition in literary analysis is about abstraction: recognizing that seemingly different character actions are all examples of the same underlying motivation or psychological pattern.
Events	A specific trigger that activates the system from a waiting state to an active response state.	A reader typing a keyword and pressing Enter triggers the chatbot's response. Before this event, the chatbot waits silently. The event starts the entire response process.	Narrative turning points are events: a single moment that triggers an irreversible change in a character's path. Identifying these moments reveals the author's structural choices.

Conditionals	IF a condition is true, THEN perform action A; otherwise, perform action B — allowing branching behavior.	The chatbot's analytical intelligence is built from 6 to 8 if/else branches. Each branch checks whether the reader's input contains a specific keyword. The ELSE branch handles all unrecognized input in the character's authentic voice.	Character decisions follow conditional logic: IF this character values power over loyalty, THEN they make that choice. Expressing this logic with if/else statements makes the student's literary interpretation clear and testable.
Functions & Parameters	A named, reusable procedure invoked with variable inputs (parameters) that tailor its output each time.	"character_voice (theme, evidence)" is the custom Scratch block called by every conditional branch. The two parameters, the thematic topic and the textual evidence, are the analytical decisions the learner makes.	Selecting the most precise, contextually accurate quotation as the evidence parameter is the literary analysis act. Different evidence parameters produce different interpretive claims about the same character.

Project Title:

WHO AM I: BUILDING A LITERARY CHARACTER CHATBOT

Project Description:

For this project, learners will apply computational thinking concepts such as events, conditionals, functions, and parameters, along with sequences, loops, and abstraction, to design and build an interactive literary character chatbot using Scratch (scratch.mit.edu), a free, browser-based block-coding platform. The project suits students with no prior programming experience: all technical skills are introduced through a teacher-led walkthrough and reinforced with structured pair programming.

The project aims to deepen literary character analysis by transforming the traditional essay format into a dynamic, interactive design challenge. Instead of writing about a character, learners program a chatbot that speaks as the character — responding to thematic questions from readers using directly cited, contextually accurate textual evidence. The chatbot's conditional logic branches encode the learners' literary interpretations: each if/else branch represents a theme, and each branch calls a reusable character-voice function with two parameters — the theme and the evidence quotation. This design ensures that decisions in programming and literary analysis are inseparable and mutually accountable.

The project directly addresses CCSS RL.9-10.3 by requiring analysis of complex characters with multiple or conflicting motivations, and CCSS RL.9-10.1 by demanding strong, thoroughly cited textual evidence for every chatbot response. It is adaptable to any literary text and any secondary ELA course.

Project Instructions:

The instructions below cover all parts of the project development process: the prior knowledge needed, the step-by-step process for completing the project, the requirements that ensure learners use CT and

ELA skills, how instructors can support learners at each stage, and the resources and tools required. Instructions are written in clear, student-friendly language.

What Learners Need Before Starting:

Literary Knowledge: Learners should have finished reading their assigned text and have their annotated copies or reading notes ready. No prior knowledge of the character's "correct" interpretation is necessary; the project is meant for learners to find and support their own evidence-based interpretations.

Technology Access: A computer, Chromebook, or tablet with a modern browser. No downloads, installations, or paid accounts are needed. Scratch (scratch.mit.edu) is completely free and browser based.

Programming Experience: No additional requirements. All necessary Scratch skills for this project, including loops forever, if/else conditions, ask-and-wait input, and custom My Blocks functions, are introduced in Step 2 of the project sequence by a live teacher demonstration.

Step-by-Step Project Sequence

The table below outlines the complete, ordered sequence of steps learners follow to complete the project. Each step is designed so that the intellectual work of deep literary analysis comes before and guides the technical work.

#	Phase	Activity Type	Duration	What Learners Do	Instructor Support & Tips
1	Analyze the Character	Group Activity	25 min	Working in groups of 3–4, complete the Character Analysis Evidence Worksheet. Identify 6–8 thematic topics central to your character's development — not surface traits, but deeper themes tied to their motivations and contradictions (e.g., ambition, guilt, loyalty, identity, mortality, power, love, fear). For each topic: • Find 1–2 direct quotations from the text that best represent the character's voice on this theme. Write the exact quotation with a chapter or act/scene citation. • Draft the character's chatbot response in first person, naturally embedding the quotation. • Draft the character's ELSE response — what would this character say to a completely off-topic question? It must sound like the character, not	INSTRUCTOR: Circulate during group work and question weak evidence choices: "Is this the most precise quotation for this topic, or just the most convenient one?" Return worksheets with specific written feedback before Step 3. Use the Lady Macbeth model chatbot (lemarg.pencilcode.net/edit/Chatbot) as a class reference to illustrate what strong, character-authentic responses look like.

				like a generic error message. Submit the worksheet for instructor feedback before proceeding to coding.	
2	Learn the Scratch Foundation	Whole Class	10 min	Open Scratch at scratch.mit.edu. The instructor will demonstrate the 4-block chatbot engine while learners build along on their own devices: • "When the green flag is clicked," starts the chatbot. • "forever" loop keeps it listening continuously. (The instructor will delete this block temporarily so learners can see the chatbot stop after one answer. Then add it back. This is how you understand what a loop does: remove it and observe what breaks.) • "Ask [What would you like to know about me?] and wait" inside the loop — this is where the reader's question enters the program. • First "if (answer contains [keyword]) then" conditional branch. The instructor will also demonstrate how to build the character voice custom function in MY BLOCKS with two parameters: theme and evidence.	INSTRUCTOR: Build gradually and audibly label each block with its CT concept name and its literary parallel as you proceed. "This is the event block — it's triggered by the reader's question, just as a narrative turning point is triggered by a specific moment in the text." Keep the demonstration to 8 minutes; the remaining 2 minutes give learners time to catch up on their own devices.
3	Build the Chatbot	Pair Programming	30 min	Work in pairs using Pair Programming. One learner is the Driver (types in Scratch); the other is the Navigator (directs decisions and checks the Evidence Worksheet). Switch roles every 10 minutes when the instructor signals. Your chatbot MUST include: • "when green flag clicked" + "forever" loop + "ask and wait" as the chatbot engine. • A custom block called <code>character_voice</code> in MY BLOCKS with two parameters: theme and evidence. Inside the block: say (join "On the theme of " (theme)) for 1.5 secs → say	INSTRUCTOR: The Navigator's primary responsibility is to verify that each conditional branch accurately reflects the Evidence Worksheet. If the literary analysis is wrong, the programming is wrong — even if the code runs without errors. When supporting stuck pairs: • Ask them to re-read the relevant worksheet entry first. • Ask the Navigator to read the branch aloud and check it against the worksheet.

				(evidence) for 3 secs → say [character's closing phrase] for 1 sec. • At least 6 if/else conditional branches — one per thematic topic — each using the "contains" operator (NOT the equals sign) and each calling character_voice with the correct parameters from the Evidence Worksheet. • A final ELSE branch at the bottom that responds in the character's authentic voice to any unrecognized question. ⚠ IMPORTANT: Always use "contains" instead of "=" in your conditions. The equals sign requires a word-for-word exact match and will cause your chatbot to fail most reader questions.	• If the technical issue persists, address it directly. The most common error is using "=" instead of "contains" — demonstrate the difference live if multiple pairs encounter it.
4	Test, Debug, and Peer Beta-Test	Pairs + Class	20 min	First, test your own chatbot independently: • Click the green flag and type each of your 6+ thematic keywords. • Verify that each response fires correctly and sounds authentically like the character. • Test the ELSE branch by typing a completely off-topic question. Fix any errors you find before the beta-testing phase. Then, exchange your Scratch URL with a partner pair. Interact with their chatbot for 8 minutes as a reader, not a programmer, and complete the Character Authenticity Review Form: • Voice Authenticity: Do all responses genuinely sound like this character? Cite 2 that are most authentic and 1 that feels off voice, with textual evidence. • Evidence Accuracy: Are all quotations accurate and used in context? Flag any that seem	INSTRUCTOR: Distribute during beta testing and ask reviewers to justify their feedback with the statement: "You said that response feels off-voice — can you show me the passage that contradicts it?" This helps prevent feedback from being impressionistic. Collect Revision Logs to ensure that revisions are analytically justified, not just technically motivated.

				misquoted or contextually incorrect. • CT Functionality: Does the loop sustain the conversation? Do all 6+ topics respond correctly? Does the ELSE branch sound like the character? • Analytical Gap: What one thematic topic is missing that this character should have a response to, based on your reading of the text? Return the form and revise your chatbot, justifying each change on the Revision Log.	
5	Write the Reflection	Individual	10 min	Write individual responses to the following three reflection prompts: • Choose the one conditional branch in your chatbot that requires your deepest analytical decision. Explain: What is the thematic question? Which quotation did you choose as the evidence parameter? Why does this specific passage best represent your character's psychology on this theme? What does it reveal about the author's craft? • Explain how the character voice function made your chatbot more efficient as both a program and a literary analysis tool. What would your code look like without it? What does this reveal about the relationship between abstraction in programming and pattern recognition in literary analysis? • If you were to add a second character sprite that responds to your character in real time (Ophelia to Hamlet, Banquo to Macbeth, Jordan Baker to Gatsby), which programming concept would allow them to interact simultaneously? What aspect of your original	INSTRUCTOR: The reflection is the synthesis document. It is where learners demonstrate their ability to articulate how each programming decision encodes a literary interpretation. Return written feedback on all three prompts. Strong responses will connect CT vocabulary and literary vocabulary in the same sentence without prompting.

				character's development would that interaction reveal that a single character chatbot cannot?	
--	--	--	--	---	--

Differentiation & Accessibility

Every learner brings different strengths, challenges, and lived experiences to the classroom. This section provides structured guidance for instructors on supporting learners who need additional scaffolding, extending the challenge for advanced learners, and ensuring that the project is accessible to every student, including English Language Learners, students with IEPs or 504 plans, and learners working in lower-resource environments.

Supporting Struggling Learners

The scaffolds below are designed to reduce barriers without removing the intellectual challenge. The goal is to lower the technical floor without lowering the analytical ceiling; every learner should still make genuine evidence-grounded interpretive decisions about the character.

Scaffold	How to implement	What it preserves
Pre-built Scratch Starter File	Provide a Scratch starter file with the “forever” loop, “ask and wait” block, and the first conditional branch already included. Learners will add the remaining five branches and build the character_voice function from Step 2 as a model.	Learners still make all analytical decisions — which topics to cover, which quotations to select, and how to phrase the character's response. The technical foundation is in place; the interpretive work remains entirely their responsibility.
Evidence Worksheet with Pre-Selected Topics	For learners who have difficulty identifying thematic topics on their own, provide a worksheet with 8 pre-selected topics and 2–3 sample quotations for each. Learners select one quotation per topic, write a first-person response, and explain their reason for choosing it.	Choosing quotations and drafting character voices, the essential analytical skills, still fall on the learner. Topic identification is supported; interpretation is not.
Sentence Starters for Character Responses	Provide a reference card with sentence starters for each part of the chatbot response: opening (“As someone who values...”), evidence embedding (“As I once said: ...”), and closing. Learners fill in the character-specific content.	The challenge of synthesizing a character's voice remains. The structural framework supports learners who find it hard to follow academic writing conventions without sacrificing the need for literary analysis.
Reduced Branch Requirement	Allow learners who are significantly behind to build 4 if/else branches instead of 6, as	Depth of analysis is maintained for all required branches. Breadth is temporarily

	long as each branch demonstrates strong, precisely cited evidence. Document this adjustment in the assessment plan.	decreased. The rubric still evaluates evidence quality and character authenticity on all submitted branches.
Strategic Pair Assignment	When assigning pairs for programming (Step 3), assign a learner with stronger literary analysis skills to the Navigator role and a learner who is more comfortable with technology to the Driver role. This utilizes each learner's strengths in the most analytically demanding role.	Both roles stay intellectually engaged. The Navigator's literary skill directly enhances the chatbot's analytical accuracy. The Driver's technical confidence ensures the pair advances smoothly through the code.
Chunked Written Reflection	Break the three-part reflection into three separate, shorter submissions instead of one combined document. This enables learners to respond to one prompt each day over three days, with brief instructor feedback between each.	The full depth of reflection stays consistent across all three prompts. The cognitive effort required to produce a lengthy written document in one sitting is reduced.

Extending the Challenge for Advanced Learners

The extensions below deepen the intellectual and technical challenge rather than simply assigning more of the same work. Each extension builds on the project's core architecture and connects to broader literary, technological, or ethical questions.

Extension	What Learners Do	CT & ELA Depth
Two-Character Dialogue Chatbot	Add a second Scratch sprite representing a character in a direct relationship with the first Ophelia responding to Hamlet, Banquo responding to Macbeth, Jordan Baker responding to Gatsby. Using Scratch's "broadcast" and "when I receive" event blocks, the two characters respond to each other in sequence. When Character 1 finishes speaking, they broadcast a signal that triggers Character 2's response.	Introduces parallelism and event-driven communication between objects — both advanced CT concepts. Requires the learner to analyze how each character's psychology is revealed through their relationships with others, directly extending RL.9-10.3 to character interactions.
Unreliable Narrator Chatbot	Create a chatbot representing a narrator or character who is self-deceived or unreliable (e.g., Holden Caulfield, Nick Carraway, Stevens in <i>The Remains of the Day</i>). Include a "reliability_score" variable that decreases each time the chatbot contradicts itself. When the	Introduces variables, nested conditionals, and state-tracking — advanced programming concepts. Requires sophisticated literary analysis of narrative unreliability and authorial irony, expanding

	reliability score drops below 3, add a self-contradicting layer after every third response.	RL.9-10.3 to RL.9-10.6 (point of view and purpose).
Cross-Text Archetype Comparison	Create the character_voice function for two characters from different texts (e.g., Atticus Finch and Gandalf; Hamlet and Oedipus). Call the same function with different parameters for each character and compare the results. Write a brief analytical paragraph explaining what the shared function structure reveals about universal archetypes and what the different parameters show about how different authors craft character uniquely.	Shows that functions serve as a tool for recognizing patterns across texts — the highest form of literary abstraction. Links Functions and Parameters directly to the RL.9-10.5 standard on authorial structural choices.
AI Ethics Extension	Research how a real commercial chatbot (such as Siri, Alexa, or a customer service bot) handles input it does not recognize. Compare its ELSE branch logic to the literary chatbot the learner built. Write a brief response to this question: "When a chatbot gives a wrong or harmful answer, who is responsible—the builder, the platform, or the user?" Connect the experience of the literary chatbot to broader issues about AI design, bias, and accountability.	Connects the CT learning to digital citizenship and AI ethics. Develops the Questioning Computational Perspective from Brennan and Resnick (2012) — learners use their programming experience to question the technology that surrounds them instead of simply accepting it.

Accessibility Considerations

The following guidance is grounded in Universal Design for Learning (UDL) principles — designing instruction from the outset to be accessible to the widest range of learners, rather than retrofitting accommodations after the fact. The three UDL principles most relevant to this project are:

- Multiple Means of Representation — provide information in more than one format;
- Multiple Means of Action and Expression — allow learners to demonstrate understanding in more than one way;
- Multiple Means of Engagement — offer different ways to motivate and sustain learner effort.

Learner Group / Need	Accessibility Strategy	Implementation Notes
English Language Learners (ELL) / Multilingual Learners	Allow the Character Analysis Evidence Worksheet to be filled out in the learner's native language. The learner then translates their character responses into English for the Scratch chatbot input. The CT	Pair ELL learners with bilingual peers when possible during the POGIL group analysis phase. If learners are studying a text that has a version in their home language, let them cite from that version. Literary analysis

	architecture—loops, conditionals, functions—is language-independent and doesn't require English proficiency to understand or use.	skills transfer across languages; the Scratch programming skills are equally accessible in any language setting.
Learners with IEPs or 504 Plans	Flexible timing: allow additional time on all phases, particularly the POGIL worksheet (Step 1) and written reflection (Step 5). Oral reflection option: learners may respond to one or more reflection prompts verbally in a recorded voice memo or in conversation with the instructor. Reduced requirement: 4 conditional branches instead of 6, with the same rubric standards applied to all submitted branches.	Record all accommodations in the assessment plan before starting the project. The rubric criteria — character authenticity, evidence quality, and CT architecture — are equally applicable to the version with reduced requirements. The oral reflection should be evaluated using the same three prompts as the written version; the mode of expression may differ, but the expected analytical depth remains the same.
Learners with Visual Impairments	Scratch can be navigated using a keyboard and is compatible with most screen readers in its basic block-building mode. For learners who use screen readers: zoom and high-contrast display settings are available in Scratch through the Settings menu. The text-based view of Scratch code, accessible via the "See Inside" option on any shared project, is more screen-reader-friendly than the block view and presents the same logical structure.	Preview Scratch's accessibility settings on the specific device and browser the learner will use before class. The Scratch Foundation maintains accessibility documentation at scratch.mit.edu/accessibility . For learners who find block-based coding inaccessible, the instructor may offer the option to write the chatbot logic as a structured pseudocode document using the same conditional and function structure, assessed with the same rubric.
Learners with Motor / Physical Access Needs	Scratch is fully navigable using only the keyboard — all block-building can be done without a mouse by using keyboard shortcuts. For learners who find keyboard use challenging, voice input software (including in most operating systems) can be used to navigate Scratch. The Pair Programming structure naturally supports motor access	Assign the Navigator role to learners who benefit from verbal rather than physical engagement. Ensure that the learner's specific device has accessibility settings enabled before starting the project. The Navigator role is analytically demanding and fully participatory — it is not a less-responsibility accommodation.

	needs: a learner with motor difficulties can assume the Navigator role (verbally guiding decisions) while their partner acts as Driver.	
Learners with Cognitive or Processing Differences	Provide a one-page visual reference card summarizing the five Scratch blocks needed for this project with a labelled screenshot of each. Provide chunked, numbered sub-steps for each stage of Step 3 (building the chatbot) so learners can check off each micro-task as completed. The step table in the Project Instructions can be printed and used as a physical checklist.	Cognitive load is highest in Step 3 (building the chatbot). Anticipate this and circulate most actively during this phase. The visual reference card should be available throughout the project, not just introduced once. Breaking the "build" phase into clearly defined micro-tasks (build the engine, build one branch, test it, add the next branch) reduces the cognitive demand of holding the full architecture in working memory.
Technology Equity / Low-Bandwidth Access	Scratch works well in low-bandwidth environments and on older devices. If internet access is unreliable, the Scratch Desktop application (a free offline download) offers the same features without needing a persistent connection. Sharing projects offline: learners can save their Scratch project as a file (.sb3 format) and submit the file directly if URL sharing isn't available.	Download the Scratch Desktop installer in advance and store it on a USB drive or a school server if internet access is unreliable. The .sb3 file format works with both the online and offline versions of Scratch. All linked tutorial videos should be downloaded and made available offline if bandwidth is limited. Ensure that all linked videos have closed captions enabled for learners who are Deaf or hard of hearing.
Deaf or Hard-of-Hearing Learners	Confirm that all linked video resources (Scratch tutorial videos cited in the Resources section) have closed captions enabled before assigning them. In class, all verbal instructions given during the teacher-modeled Step 2 walkthrough should be accompanied by projected written step-by-step text so that learners do not rely solely on audio. The step table in the Project Instructions	Closed captions should be verified, not assumed. Auto-generated captions on YouTube vary significantly in accuracy; manually verified captions are preferable. The structured written step table in the Project Instructions is the primary accessibility support for this group — it means that every verbal instruction has a written equivalent that learners can reference independently throughout the project.

	serves as a written reference for every verbal instruction given.	
--	---	--

Project Requirements Checklist

The following requirements ensure that learners apply their programming knowledge and literary analysis skills together in every component of the project:

#	Requirement	Why it matters
1	Character analysis Evidence Worksheet completed before coding begins	Ensures that analytical work guides programming. A learner who codes before analyzing creates a technically functional but intellectually shallow chatbot.
2	At least six if/else conditional branches, each based on a directly quoted, cited passage	Ensures a broad character analysis. Six topics require engaging with the entire text, not just one memorable scene.
3	Every branch calls <code>character_voice</code> using the "contains" operator with both parameters.	Ensures functions and parameters are properly implemented and that the evidence parameter reflects a valid analytical decision.
4	A character-accurate ELSE branch for unrecognized input	Ensures deep character understanding — speaking as a character when asked an off-topic question requires knowing them thoroughly.
5	Peer Beta Testing concluded with a Character Authenticity Review Form.	Fosters genuine accountability and peer learning. Inaccurate chatbots are identified and corrected based on evidence from reader feedback.
6	Written Reflection addressing all three prompts	Ensures learners can clearly explain how their programming choices relate to their literary interpretations — the key transferable skill of this project.

Resources & Tools

Resource	Details & Access
Programming Platform	Scratch scratch.mit.edu Free, browser-based block coding environment. No download, installation, or paid account needed. Compatible with Chrome, Firefox, Safari, and Edge, including Chromebooks and tablets. To share a project: click "Share" and copy the URL from the browser address bar. Submit this URL as the final chatbot deliverable.
Model Literary Chatbot	Lady Macbeth Chatbot lemarg.pencilcode.net/edit/ChatBot A working literary chatbot built by Dr. Lauren Margulieux (course Week 8 resource), programmed as Lady Macbeth. Instructors and learners should interact with this model before beginning the project to calibrate what a strong chatbot response looks and sounds like. Note: this model uses Pencil Code rather than Scratch, but the underlying event-conditional-function logic is identical.
Scratch Help	The Scratch Test Panel (bottom-right of the editing screen) allows learners to try any command immediately and see its output — ideal for experimenting with "contains" vs. "=" before using either in a conditional branch. Type any block name in the Test Panel to get built-in help text.

Scratch Tutorial (Functions)	For learners who want additional support building the character_voice custom function: Scratch Custom Blocks — Beginner's Guide (video). For functions and parameters in Scratch specifically: Parameters in Scratch (video).
Evidence Worksheet	Distribute the Character Analysis Evidence Worksheet before Step 1. The worksheet should include character name, 6–8 thematic topic rows, columns for "Direct Quotation + Citation," "First-Person Character Response Draft," and "ELSE Response Draft." Instructors may design this as a Google Doc, a printed handout, or a shared digital template.

Project Assessment:

Students submit three components that together constitute the complete project portfolio. The final artifact is a working, interactive Scratch literary character chatbot. The three components are designed to be mutually accountable: a strong Scratch program with weak worksheet analysis, or a strong worksheet with a weak chatbot, each reveals a gap that the rubric identifies and addresses.

#	Deliverable	What the Final Product Looks Like
1	Scratch Chatbot URL	A working Scratch chatbot that, when the green flag is clicked, runs a continuous forever loop, accepts reader questions via "ask and wait," evaluates each question through 6–8 if/else branches using "contains," calls character_voice(theme, evidence) in every branch, and responds with character-authentic, evidence-grounded dialogue. Off-topic questions trigger a character-authentic ELSE response. The program runs without errors and is shareable via URL.
2	Character Analysis Evidence Worksheet	A completed pre-programming analytical document showing: 6–8 thematic topics with direct quotations and full citations; first-person character response drafts for each topic; and a justified ELSE response draft. This document is the analytical blueprint of the chatbot and is assessed independently for analytical depth.
3	Written Reflection (3 prompts)	Individual written responses to all three reflection prompts demonstrating that the learner can connect each programming decision to its literary interpretation. Strong reflections use CT and literary vocabulary together in the same analytical sentence.

Assessment Rubric

Criteria	Excellent (4)	Proficient (3)	Developing (2)	Beginning (1)
Chatbot CT Architecture (Scratch Program)	All CT elements are correctly implemented: the forever loop, ask-and-wait, 6–8 if/else branches using "contains,"	Most CT elements are present: loop, conditionals, and function implemented. 1–2 branches bypass the function or use "="	Partially built; missing loop or function; fewer than 5 branches; some responses are hardcoded; program may freeze	Incomplete; fewer than three working branches; no custom function; loop missing; CT concepts not

	character_voice function with both parameters called in every branch, and a character-authentic ELSE; the program runs without errors.	instead of 'contains.' The program runs with minor issues.	or unexpectedly stop.	meaningfully integrated.
Character Authenticity & Textual Evidence	All 6–8 responses clearly reflect the character's voice; each response includes an accurately quoted, in-context passage with full citation; evidence choices are precise and can be justified analytically.	Most responses character-authentic; evidence present and mostly accurate; 1–2 responses slightly off-voice or evidence choices could be more precise; citations mostly complete	Some responses are generic instead of character-specific; evidence is included but sometimes paraphrased; the context of 1–2 quotations are questionable; citations are incomplete.	Responses are generic — indistinguishable from a plot summary; evidence missing, paraphrased, or incorrect; no identifiable character voice.
Functions & Parameters (Technical + Analytical)	character_voice defined with both parameters; all branches call it consistently; no hardcoded responses; reflection explains both how the function works and how parameter selection represents a literary argument.	Function is defined and called in most branches; one branch is hardcoded; reflection describes the function but not its analytical importance.	Function is defined but called inconsistently; mixes between function calls and hardcoded text; reflection indicates only partial technical understanding.	Function exists or is defined but is never invoked; all responses are hardcoded; shows no clear understanding of functions or parameters.
Evidence Worksheet (Analytical Depth)	6–8 themes are subtle yet thematically important; quotations are precise, accurate, and well-contextualized; first-person drafts show real character understanding; ELSE response is authentic and justified.	Six or more themes are appropriate and relevant; quotations are mostly accurate; drafts mostly authentic to characters; other elements are present but somewhat generic.	Limited to fewer than six themes or surface-level trait options; some quotations are paraphrased; drafts come across as generic; ELSE is missing.	Worksheet incomplete; themes unclear or superficial; quotations missing or inaccurate; little analytical engagement evident.
Reflection (Synthesis)	Sophisticated link between programming choices and literary interpretation in each prompt; uses	Strong literary connection across most prompts; vocabulary generally precise; all three prompts	Surface-level connections are stated but not explained; one prompt is underdeveloped; CT	Reflection is absent or vague; there is no clear link between programming choices and literary

	CT and literary vocabulary together analytically; identifies at least one transferable application beyond this text.	sufficiently developed.	and literary vocabulary are used separately rather than together.	interpretation; CT vocabulary is missing or misused.
--	--	-------------------------	---	--

Active Learning Approach:

This project is based on the idea that learners gain a deeper, more transferable understanding when they build knowledge through doing, rather than passively observing. Research consistently shows that learners in active learning settings perform better than those in traditional lecture environments: a meta-analysis of STEM courses revealed that students in lecture-based classes were 1.5 times more likely to fail than their counterparts in active learning settings (Freeman et al., 2014). The "Who Am I?" project incorporates four evidence-based active learning strategies from the course literature (Ericson, 2023), each implemented at a specific stage of the project where it has the greatest impact on learning.

Strategy	How it is used	Student Role	CT / ELA Connection
Peer Instruction (Ericson, 2023)	The project opens with a live demonstration of a model chatbot. The instructor poses a conceptual question: "What does this chatbot reveal about this character that a plot summary cannot?" Learners write an individual answer (First Vote), discuss with a partner for 2 minutes, then share with the class (Second Vote).	Learners actively surface prior knowledge, form initial interpretations, test them against a partner's reasoning, and revise. They arrive at the key insight — that every chatbot response encodes a literary interpretation — through reasoning rather than instruction.	Establishes the intellectual framing that motivates the entire project. Connects Events and Conditionals (CT concepts) to textual evidence and character voice (ELA). Directly supports RL.9-10.3.
POGIL-Style Group Inquiry (Ericson, 2023)	Groups of 3–4 complete the Character Analysis Evidence Worksheet collaboratively using a structured inquiry protocol before any coding begins. The worksheet guides groups through topic identification, quotation selection, first-person response drafting, and ELSE-response reasoning.	Learners construct analytical understanding together, challenge each other's evidence choices, and produce a shared interpretive document that will drive their individual programming. The worksheet is the analysis blueprint, not a warm-up activity.	Ensures that literary analysis precedes and drives programming — the analytical work is the intellectual center of the project. Directly supports RL.9-10.1 (cite strong evidence) and RL.9-10.3 (analyze complex character motivations).
Pair Programming (Driver / Navigator)	Learners work in pairs with assigned, rotating roles:	Both roles are analytically demanding: the Driver must	Reinforces iterative CT practice (Brennan &

	the Driver types in Scratch, while the Navigator makes decisions and reviews the Evidence Worksheet. Roles switch every 10 minutes upon instructor signal.	make decisions accurately; the Navigator needs to verify that every conditional branch is correct before the Driver codes it. Neither role is passive.	Resnick, 2012). The Navigator role distributes literary accountability between the pair — the chatbot is technically correct only if the literary analysis is correct. Develop communication and collaboration skills.
Peer Beta Testing (Interactive ebooks model — Ericson, 2023)	Learners share Scratch chatbot URLs and interact with a partner's chatbot as a reader for 8 minutes, filling out a structured Character Authenticity Review Form. Feedback must be based on direct textual evidence.	Learners receive immediate, specific, evidence-based feedback on their chatbot's character authenticity and CT functionality from a real reader — not from automated grading. They must justify every revision on a Revision Log.	Creates authentic audience accountability: a chatbot that sounds generic or misrepresents the character is quickly identified. Models the Testing and Debugging CT practice (Brennan & Resnick, 2012) and the Connecting Computational Perspective—creating for an audience that responds.

References:

- Brennan, K., & Resnick, M. (2012). New frameworks for studying and assessing the development of computational thinking in interactive media design. Paper presented at the Annual Meeting of the American Educational Research Association, Vancouver, BC, Canada.
- Common Core State Standards Initiative. (2010). English language arts standards — Reading: Literature Grades 9–10 (RL.9-10.1, RL.9-10.3). National Governors Association and Council of Chief State School Officers.
- Ericson, B. (2023). Four ways to add active learning to computing courses. *Communications of the ACM*, 66(2), 26–29. <https://doi.org/10.1145/3576930>
- Freeman, S., Eddy, S. L., McDonough, M., Smith, M. K., Okoroafor, N., Jordt, H., & Wenderoth, M. P. (2014). Active learning increases student performance in science, engineering, and mathematics. *Proceedings of the National Academy of Sciences*, 111(23), 8410–8415.
- International Society for Technology in Education. (2016). *ISTE Standards for Students*. ISTE.
- Krauss, J., & Prottzman, K. (2016). *Computational thinking and coding for every student: The teacher's getting-started guide*. Corwin Press.
- Margulieux, L. (n.d.). ChatBot — Lady Macbeth [Pencil Code program]. Retrieved from <https://lemarg.pencilcode.net/edit/ChatBot>
- MIT Media Lab — Lifelong Kindergarten Group. (n.d.). Scratch [block-based programming environment]. Retrieved from <https://scratch.mit.edu>